CS 5433 SP24: Homework 4

Instructor: Ari Juels
TAs: James Austgen, Arkaprabha Bhattacharya, Benjamin Chan
Due: 7 May 2024

In the previous homework, we got our hands dirty with Solidity and smart contracts. In HW4, we are going to learn about smart contract security by hacking some vulnerable smart contracts!

Policies

Submit to CMSX.

Integrity. For all assignments, you may make use of published materials, but you must acknowledge all sources, in accordance with the Cornell Code of Academic Integrity. Additionally, you must ensure that you understand the material you are submitting; you must be able to explain your solutions to the course instructor or TA if requested. You must complete this homework assignment *on your own.*

Disclaimer

Please start the assignment early. We do not guarantee responses from the TAs or instructors on errors in the assignment or broken packages without at least 48 hours' lead time.

1 Instructions

Your task in this assignment is to "hack" the contracts in the given code. In the last homework, you've already familiarized yourself with using Remix and Metamask. In this homework, you will get your hands dirty.

This homework assignment is primarily driven by the HackThisContract website, which you can access here: `http://3.141.11.62/`. This website will deploy vulnerable contracts for you to hack. All deployments are connected to your Ethereum address, and the associated contract code/addresses are also presented on the site.

Again, you will need to get Holesky testnet ether. Feel free to use the same Ethereum account you used for HW3. You can use our Ed discussion faucet (make a comment with an address you haven't submitted here before if you need extra) or the PoW faucet.

Interacting with the contracts: You can use Remix with Metamask as the "Injected Provider" to interact with the deployed HackThisContract contracts and any contracts you write yourself when trying to hack the provided contracts. You can also use other methods to interact with the Ethereum blockchain like the Hardhat, Web3.js, or Web3.py frameworks.

Note: If you are testing in a Remix VM environment, make sure to use "Remix VM (Cancun)", the latest version of the EVM. The newest versions of the Solidity compiler make use of the PUSH0 opcode, which pre-Cancun versions of the EVM don't support.

2 Grading Policy

The points for each exploit are provided below. **13 points** are required for full credit. Completing all the challenges successfully will award you **1** point of extra credit. This homework does not include partial credit.

3 Challenges

First, be sure to include the Ethereum address you are using in the challenges (i.e., your MetaMask address) in `my_addr.txt` in the root folder of your submission.

For each challenge:

- Deploy the smart contract by clicking the Deploy button on the HackThisContract website.
- Solve the challenge.
- Click the "Update Status" button to run the autograder on the problem. When a challenge's status is "Done!", that means that you have completed the problem successfully.
- Describe the sequence of steps you took to hack the contract in `[name]_steps.txt`, where [name] is the name of the challenge, e.g., `naive_programmer_steps.txt`. This should include the transactions you used to interact with the contract, and any code (if any) you used. Do not include the provided code in this file.

3.1 Hello_World

Description: A warm up contract to get your juices flowing. Say hello to it by calling the `helloWorld` function with your net ID and the magic flag. Impersonate the contract owner!

Score & Level: 1.

3.2 Naive_Programmer

Description: A commit-reveal election contract with a vulnerability. A commit-reveal election scheme involves voters committing to a vote, and then revealing their commitments (their votes) after the voting period is over. In this case, the election is over and the contract reports that Alice has won. You will have succeeded when you make the contract believe Bob has won instead. Interact with the deployed contract so that `currentWinner` is marked as "Bob".

Hacked condition: `currentWinner` returns "Bob".

Score & Level: 3.

3.3 Coin_Flip

Description: This contract implements a simple coin flip game between you and the house (player 2). You job is to violate this lottery so that you (player 1), win. (We won't count re-deploying the contract until the win condition is satisfied as a valid solution.) We recommend writing a script (ethers.js or web3.py are good choices) that interacts with the contract automatically.

Hacked condition: Player 1 is the winner of the coin flip with 100% success probability.

Score & Level: 3.

3.4 Lost_Ether

Description: A programmer has made an ERC20 with a silly vulnerability. Your job is to violate one of the invariants of the contract: an agent sends money to the contract and doesn't get any tokens back.

Hacked condition: The balance of the contract address is not equal to the `totaleth()`.

Score & Level: 1.

3.5 Wrapped_Ether_Black_Hole

Description: A simple ERC20 wrapped ETH contract, which lets you mint tokens with ETH and burn them to receive the ETH back. It's your job to trap the ETH in the contract forever and prevent anyone from burning their tokens!

Hacked condition: All calls to `burn()` revert because `isConsistent()` returns false.

Score & Level: 3.

Hint: The solution won't involve calling any of the smart contract's functions. There are two feasible ways of doing this (and a third which would require you to be a validator on the chain). Feel free to search online for ideas.

3.6 Bad_ICO

Description: A greedy programmer has been impatient and created an ERC-20 ICO without any thought. Your job, again, is to violate one of the invariants of this contract: an agent sends money to the contract and doesn't get any tokens back.

Hacked condition: The balance of the contract address is not equal to the `totaleth()`.

Score & Level: 2.

3.7 Multisig_Magic

Description: A vulnerable multi-signature wallet contract. A naive user has deposited 0.005 Ether in this contract, and it's your job to make it yours. Your attack will have succeeded when the contract's balance falls below this initial 0.005 ETH.

The wallet contract *delegates* calls to the wallet library contract when it doesn't have a function that matches the one in the call. Such a delegated call runs the code of the wallet library contract in the context of the wallet contract (i.e., any storage reads/writes done by the wallet library's code will read/write from the wallet contract).

Note: Make sure to select the correct, much older version of Solidity from the version dropdown in the Compiler tab of Remix.

Hacked condition: The balance of the contract address is less than 0.005 ETH.

Score & Level: 3.

Hint: This is a "real" smart contract that got hacked in 2017.

4 Tips

Here are some tips based on feedback from previous years' students:

- Make sure that the compiler you use in Remix matches the header at the top of each file.
- You can interact with existing smart contracts in Remix using the "At address" field in the "Deploy & run transactions" tab.
- Here is a series of YouTube videos about using Remix that may be helpful for this assignment:

  https://www.youtube.com/watch?v=xxJfQJ5bMfw&list=PLbbtODcOYIoEOD6fschNU4rqtGFRpk3ea&index=2&pbjreload=10
- **In Remix, make sure you are using the "Injected Web3" Environment when calling smart contract functions or it will not point at the Holesky network!**
- When using the Injected Web3 environment, be sure to approve Remix's connection to your wallet inside MetaMask. The dropdown list of accounts should contain your Ethereum address.

Submission Instructions

In addition to the files listed below, please submit any additional code you wrote for the challenges (Solidity, JS, Python, etc). Do not resubmit the provided code.

```
hw4-<NetID>
├── my_addr.txt
├── Hello_World
│   └── hello_world_steps.txt
├── Naive_Programmer
│   └── naive_programmer_steps.txt
├── Coin_Flip
│   └── coin_flip_steps.txt
├── Lost_Ether
│   └── lost_ether_steps.txt
├── Wrapped_Ether_Black_Hole
│   └── wrapped_ether_black_hole_steps.txt
├── Bad_ICO
│   └── bad_ico_steps.txt
└── Multisig_Magic
    └── multisig_magic_steps.txt
```